

DTS 201 · LECTURE 02

Data, Data Types & Data Sources

Learning Objectives

- Differentiate between data, a data point, and a dataset
- Decide which observations actually matter to an analysis
- Define structured, semi-structured and unstructured data
- Classify data types by discrete, continuous, nominal, ordinal
- Use pandas to load a CSV and inspect it

01

What is data?

Data is a raw observation

Data refers to the raw observations that we take note of, record, or store.

When you put together pieces of observations that are relatable (that is, under the same subject) it is called a **dataset**. When a good dataset is well analysed it produces reliable information.

So in analysis, data is what we use to establish facts and derive the insights that guide decisions.

videos

photographs

voice notes

text messages

documents

web pages

spreadsheets

sensor readings

transaction logs

A data point

One single observation about one subject.

Key Characteristics

- **Single unit:** It represents one specific value, word, image, or measurement.
- **Building block:** Millions of data points combine to form a complete dataset for analysis.
- **Distinct:** It must be recognizable and separate from other pieces of information.

For example:

1. The number of words in an email I wrote to the HOD yesterday. *I am the subject and the single observation here is the number of words in this specific email.*
2. The height of Mr. Bean. *Mr Bean here is the subject and the only think we are observing about him, is his height.*

Data point is simply a single value that is attached to a single subject we observed.

Data

Data is formed when several data points are used to describe one subject.

For example:

The email I sent to the HOD had 312 words, and I sent it at 2:14 pm WAT. There were also 7 links in that email and this wasn't my first time emailing him.

A3	▼	fx					
	A	B	C	D	E	F	G
1	sender name	receipient	number_of_words_in_email	time_sent	number_of_links	known_sender	
2	Sharon	HOD	312	2:14	7	Yes	
3							
4							
5							
6							
7							

Each data point is a value of a **feature** (also called **attribute**, **variable** or **field**). In this record, there are 6 major attributes of the email. Since this belongs to just one subject, they will be stored in the same row.

The column headers are called features, while the rows are called samples (which we were referring to initially as subjects)

A dataset

In a dataset many subjects are covered and it is the most useful because it allows comparison.

Comparison is what makes data useful and analysis possible.

A6

▼

fx

Marya...

	A	B	C	D	E	F	G
1	sender name	receipient	number_of_words_in_email	time_sent	number_of_links	known_sender	
2	Sharon	HOD	312	2:14	7	Yes	
3	Tessy	Augusta	113	4:01	0	No	
4	Olumide	Dr Ogbonna	268	9:23	0	Yes	
5	Alfred	Eke	3	11:54	1	Yes	
6	Marya...						
7							
8							

A dataset is an organized collection of related data usually stored in a table, spreadsheet, or file.

Every action generates data

For example, sending a single email produces all of this, and even more.

Number of words	Subject line	Time of day sent	Anyone on Cc or Bcc
Device it was sent from	Time spent in drafts	Sender's location	Receiver's location
Prior contact between them	URLs in the body	Special characters, emojis	How the email closes
Attachment count and type	Reply-to differs from sender	Capital letters in subject	... and dozens more

The question is do we need every possible data point in our analysis?

Focus on the detail that matters

We have the liberty to record every observation but that does not mean we should.

When a single subject carries a large number of data points, analysis gets harder. Now imagine when we now have thousands or millions of subjects

The discipline is choosing observations that produce the detail that matters to *your* question.

WHAT TOO MANY COLUMNS COSTS YOU

- Too much time and resources spent in collecting and storing the data
- It makes the data overwhelming, resulting to analysis fatigue, which might limit your insight on the actual thing.
- Slower, heavier processing
- Harder to explain your findings

The next class should cover principles of data collection

02

Data formats

Data comes in these three forms

01

Structured

Fixed rows and columns. Every record has the same fields, in the same order.

For example: CSV, SQL tables, Excel Spreadsheets

02

Semi-structured

It has labels and hierarchy (such as keys, tags, index, etc), but it has no fixed table like the structured form. Records may differ from each other.

For example: JSON, XML, emails, logs, etc

03

Unstructured

No labels at all. Meaning data points have to be extracted before analysis can begin.

For example: images, audio, video or free text

Structured data

Data is structured when a quick glance tells you what it contains. This is because the labels and the shape are already fixed.

	A	B	C	D
1	student	age	score	grade
2	Amara	21	88.5	A
3	Bode	24	71.0	B
4	Chidi	22	64.2	C
5	Zainab	20	93.8	A

■ Header row — names each field □ One row = one record

HOW TO RECOGNISE STRUCTURED DATA

- Every record has the same fields
- Each column holds one type of value
- You can sort, filter and total it immediately
- Ready for pandas with no preparation

Semi-structured data

It carries labels, so like the structured, it is easy to find things but the records are free to differ, and values can nest inside other values.

```
[{
  "student" : "Amara",
  "age" : 21,
  "phone" : "0801-000-0000",
  "scores" : [88.5, 74, 91]
},
{
  "student" : "Bode",
  "address" : {
    "city" : "Lagos",
    "area" : "Yaba"
  }
}]
```

- Amara has age and phone attributes; Bode does not. This means that fields are not fixed
- **scores** is a list; and **address** is a record inside a record
- To get a **structured** table, it must be flattened first

Unstructured data

No fields, no labels, no rows. The information is in there, but something or someone has to pull it out before analysis can start.



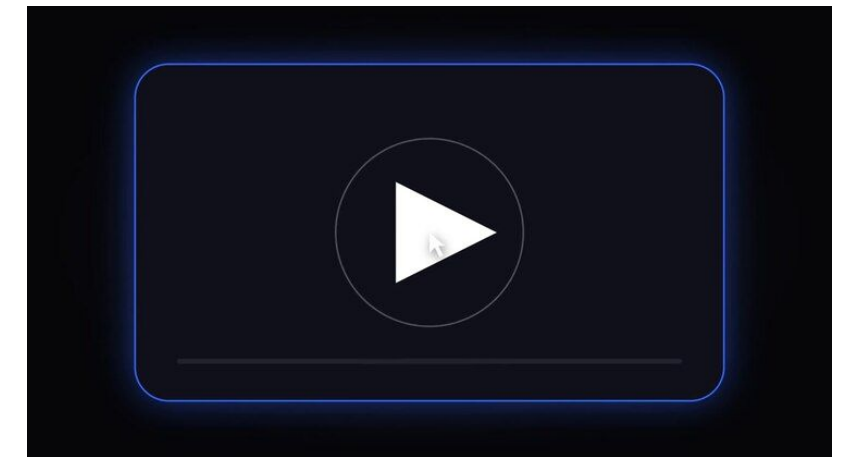
Pixels in the picture can be extracted to get the colour shades.



A waveform. But we can extract the transcript, audio length, etc.

"Delivery guy came two days late and the box was open. This is not how to run a business at all"

A Free text but we can extract the sentiment, length of text, etc.



We can extract the individual frame that make up the video.

Summary

	Structured	Semi-structured	Unstructured
LABELS	Fixed columns	Keys, but variable	None
TYPICAL FILES	.csv .xlsx, SQL	.json .xml, logs	.jpg .mp3 .mp4 .txt
PREP NEEDED	Little	Parse and flatten	Extract features
SHARE OF DATA	the minority	growing fast	the vast majority

03

Data types

NUMERICAL / QUANTITATIVE

Values you can do arithmetic on

CATEGORICAL / QUALITATIVE

Values that place a subject in a group

Discrete

Countable values.
Examples are; students in a class, links in an email, cars sold

Continuous

Measured on a scale.
Examples are: height, weight, temperature, delivery time

Nominal

Named groups with no natural order. For example: city, gender, blood group, payment method

Ordinal

Groups that rank. For example: grade A-F, satisfaction 1-5, income band

04

Intro to pandas

What is pandas?

pandas is a Python library that is for working with data in tables. The name comes from *panel data*, a statistics term for tables observed over time.

Code Convention: `import pandas as pd`

Think of it as a spreadsheet you operate with code instead of a mouse clicks. You can load a file, look at it, clean it, filter it, group it, and export it.

WHAT YOU WILL USE IT FOR

- Reading files such as CSV, Excel, JSON, SQL
- Inspecting shape, types and missing values
- Cleaning: renaming, filling, dropping, converting
- Selecting rows and columns by condition
- Grouping and summarising
- Exporting to another format

Series and DataFrame

SERIES (ONE COLUMN)

A single labelled column of values. It has an index, the column values and a dtype.

index	score
0	88.5
1	71.0
2	64.2
3	93.8

dtype: float64

DATAFRAME (MANY COLUMNS)

A DataFrame is basically a table or a several Series lined up on the same index.

index	student	age	score	grade
0	Amara	21	88.5	A
1	Bode	24	71.0	B
2	Chidi	22	64.2	C
3	Zainab	20	93.8	A

pandas dtype

Data type	pandas dtype	What to look out for
Discrete	<code>int64</code>	One missing value in a column of integers turns the entire column into <code>float64</code>
Continuous	<code>float64</code>	Numerical values stored as text would have the object datatype
Nominal	<code>object</code> / <code>category</code>	Never encode as 1, 2, 3 without one-hot
Ordinal	<code>category (ordered)</code>	You must state the order; pandas will sort A-Z otherwise
Date / time	<code>datetime64</code>	Read as text unless you parse it explicitly

05

Lab session

Set up:



If you can't scan, [use this URL](#)

[Notebook used in class](#)

Step 1: Scan the QR Code below. It should take you to a drive folder called “**Datasets**”

Step 2: Right click on “Datasets”, then select Organise and Click on “Add Shortcut”.

You are doing this because Colab cannot allow you access anything that is not in your drive. Since I shared this with you, it would naturally be stored under “Shared with me”, and you won’t be able to access the file from Colab.

Step 3: Choose the path where you want to store “Datasets”. You can store it directly in “My Drive” or create a new folder inside My Drive called “DTS 201” and store the Datasets folder there.

Step 4: After storing the folder, create another folder inside “DTS 201” called “Notebooks”.

Step 5: Open the “Notebooks folder and create a Google Colaboratory notebook for this lab session.

LAB 01 Load a CSV

Our file: `50_startups.csv`

CELL 1

```
import pandas as pd

df = pd.read_csv("50_startups.csv" )
df.head()
```

USEFUL VARIATIONS

```
pd.read_csv(url)                # straight from the web
pd.read_csv(path, sep=";")       # other separators
pd.read_csv(path, nrows=100)     # peek at a big file
df.tail(3)                      # last three rows
```

WHAT JUST HAPPENED

- pandas read the file and returned a **DataFrame**
- The first line became the column names
- Rows got a default index of 0, 1, 2, 3...
- pandas would guess the dtype for every column

IF IT ERRORS

FileNotFoundError means the path is wrong. In Colab, upload the file first, then use the name exactly as it appears in the file panel.

Series vs DataFrame

CELL 2

```
s = df["score"] # Series
d = df[["score"]] # DataFrame
print(type(s)) print(type(d))
print(s.shape, d.shape)
```

```
<class 'pandas.core.series.Series'>
<class 'pandas.core.frame.DataFrame'>
(4,) (4, 1)
```

READ THE DIFFERENCE

Series

One column. Shape (4,)
i.e a single number, so one dimension. Has one dtype.

DataFrame

A table. Shape (4, 1) —
rows and columns. Has
dtypes, plural.

```
s.mean() # works on a Series
s.value_counts() # counts per value
df[["age", "score"]] # a DataFrame
df.columns # the column labels
df.index # the row labels
```

RULE OF THUMB One bracket asks for a column and returns a Series. Two brackets ask for a list of columns and returns a table.

LAB 03 Inspect your data

CELL 3 — `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   student     4 non-null      object
1   age         4 non-null      int64
2   score       3 non-null      float64
3   grade       4 non-null      object
dtypes: float64(1), int64(1), object(2)
```

CELL 4 — `df.describe()`

	age	score
count	4.00	3.00
mean	21.75	74.57
std	1.71	12.33
min	20.00	64.20
50%	21.50	71.00
max	24.00	88.50

WHAT TO LOOK FOR

- `score` is 3 non-null out of 4 — one value is missing
- `object` means text: nominal or ordinal columns
- `describe()` skipped the text columns entirely

RUN THESE TOO

```
df.shape      # (rows, cols)
df.dtypes    # type per column
df.isnull().sum() # missing per col
df["grade"].unique() # the categories
df["grade"].value_counts()
df.describe(include="all")
```

LAB 04 Moving between data forms

The CSV is structured. One method turns it into the semi-structured form we saw earlier.

CELL 5 — STRUCTURED → SEMI-STRUCTURED

```
df.to_json("students.json" ,
          orient=
"records", indent=2) # and straight back again back =
pd.read_json("students.json")
```

OTHER DESTINATIONS

```
df.to_csv("out.csv", index=False)
df.to_excel("out.xlsx", index=False)
df.to_dict(orient="records")
df.to_string()
# plain text pd.json_normalize(nested) # flatten
```

RESULT — STUDENTS.JSON

```
[
  {
    "student" : "Amara",
    "age" : 21,
    "score" : 88.5,
    "grade" : "A"
  },
  {
    "student" : "Bode",
```

NOTE Converting the form does not change the data types — age is still discrete, grade still ordinal.

```
"score" : 71.0,
```

Guided class work

Work in your own notebook on `50_startups.csv`. Answer in markdown cells under each result.

- 01 Load the CSV and print its shape. How many rows and columns do you have?

- 02 Run `info()`. List every column and label its data type: discrete, continuous, nominal or ordinal.

- 03 Which column has missing values, and how many? Write one sentence on how you would handle it.

- 04 Pull one column out as a Series and one as a single-column DataFrame. Print both types and shapes.

- 05 Use `describe()` on the numeric columns. Report the mean score and say whether any value looks out of place.

- 06 Export the DataFrame to JSON with `orient="records"`, open the file, and describe what changed about its form.

- 07 Name three data points a university could record about a student that would *not* help predict their grade. Explain why.